NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING

Midterm (2016-17)

CS3230: Design and Analysis of Algorithms

September 2016 Time Allowed: 90 Minutes

INSTRUCTIONS TO CANDIDATES

1. This midterm paper consists of **THREE (3)** questions and **ELEVEN (11)** printed pages including this page and two extra sheets of scratch paper. You may detach the scratch paper for use during the midterm.

2. Answer all questions. You have 90 minutes to earn 40 marks.

3. Write your answer in the space provided. If you need more space, write on the back of the sheet and indicate clearly that that the solution is being continued on the back of the sheet.

4. Do not spend too much time on any problem. Read them all through first and attack them in the order that allows you to make the most progress.

5. Show your work, as partial credit will be given. You will be graded on the correctness and efficiency of your answers, and also on your clarity. Be clear and always explain your reasoning.

6. This is a **CLOSED BOOK** midterm. You are allowed to bring in one handwritten (on both sides) A4 crib sheet and use an approved calculator.

For Examiner's Use Only		
	Max Marks	Earned Marks
Problem 1	22	
Problem 2	8	
Problem 3	10	
TOTAL:	40	

Student number:

Problem 1. Short Problems (22 Marks)

Solve each of the following problems and justify your answer.

a) Solve the following recurrence.

$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(\ln\ n)$

Solution:

We apply Master Theorem case 1. To verify that case 1 applies, note that $\ln n = O\left(n^{\lg 2-\epsilon}\right) = O(\ n^{1-\epsilon})$. Applying case 1, we get $T(n) =\ \theta(n)$.

b) Solve the following recurrence.

$T(n) = 4T\left(\frac{n}{3}\right) + \Theta(n^3)$

Solution:

We apply Master Theorem case 3. To verify that case 3 applies, note that

$n^3 =\ \Omega(n^{log_3 4+\epsilon})$ and that the regularity condition also holds,

$4(\frac{n}{3})^3 =\ \frac{4}{27}n^3 < cn^3$ for $c < 1$. Applying case 3, we get $T(n) =\ \Theta(n^3)$.

c) Let $T(n) = T(n-1) + \lg n$. Prove that $T(n) =\ O(n \log n)$.

Solution:

We use the substitution method. Assume as inductive hypothesis that

$T(k) < ck \lg k$ for $k < n$. Then

$T(n) = T(n-1) + \lg n$
$< c(n-1)\lg(n-1) + \lg n$
$< c(n-1)\lg n + \lg n$
$= cn\lg n - c\lg n + \lg n$
$< cn\lg n$

for $c > 1$. The value c can also be selected to be large enough for small n for the base case to hold. Hence $T(n) = O(n \lg n)$.

Alternatively, assume as inductive hypothesis that

$T(k) < c \lg(k!)$ for $k < n$ for some value $c > 1$. Then

$$
\begin{aligned}
&T(n) = T(n-1) + \lg n \\
&< c \lg(n-1)! + \lg n \\
&\leq c \lg(n-1)! + c \lg n \\
&= c \lg n!
\end{aligned}
$$

The value c can also be selected to be large enough for small n for the base case to hold. Hence $T(n) = O(\lg n!) = O(n \lg n)$ by Stirling's approximation.

d) Describe a method for finding all the $n/3$ smallest elements of an array $A[1..n]$. You should give the asymptotically fastest method that you know of.

Solution:

Run linear time select to select the element of order $n/3$ in $O(n)$ time. Then run Partition in $O(n)$ time to get the smallest $n/3$ element.

e) Argue that an algorithm that takes $\Theta(\lg n)$ time to find an element in a **sorted** array in the comparison model of computation is asymptotically optimal.

Solution:

As there are n elements, the algorithm will have at least n leaves when represented as a binary tree. A binary tree with n leaves has height of at least $\lg n$ giving a worst case runtime of $\Omega(\lg n)$. At the same time, binary search can be used to find the correct element in time $O(\lg n)$. Hence the runtime is $\Theta(\lg n)$.

f) **True** or **False**. Sorting n integers, where each of the integers can be represented using $\Theta(\mathrm{n} \lg n)$ bits, can be done in $O(n)$ time. Justify your answer.

Solution:

We consider out linear time sorting methods. Counting sort runs in $\theta(n + k)$ where k grows at least exponentially. Using digits of size $\Theta(\lg n)$ bits, radix sort runs $\theta(n)$ iterations of counting sort where each iteration takes $\Theta(\mathrm{n})$ time. Hence

the total runtime of running radix sort in this case is $\Theta(n^2)$. Comparison sorts takes $O(n \log n)$ time, hence there is no known method for doing it in $O(n)$ time.

Comment: It would be more accurate to say that there is no known method for doing it, rather than saying that the statement is false, as pointed out by several students. Students are graded correctly for either saying false, or saying that no known linear time method exists.

g) Use a loop invariant to prove the correctness of the program Fib(n) that computes an array $F[0..n]$ consisting of the Fibonacci number F_0 to F_n below. The Fibonacci numbers are defined as $F_0 = 0$, $F_1 = 1$, and $F_n = F_{n-1} + F_{n-2}$.

Fib(n)

```
1. F[0] = 0
2. F[1] = 1
3. for i = 2 to n
4.     F[i] = F[i-1] + F[i-2]
5. return F[0..n]
```

Solution:

We use the loop invariant: $F[j] = F_j$ for $j = 0, ..., i-1$.

Initialization: The variable i has value 2. $F[0]$ and $F[1]$ has been initialized to F_0 and F_1. Hence the invariant holds.

Maintenance: Assume that the invariant holds at the start of the loop: $F[0..i-1]$ consists of the Fibonacci numbers F_0 to F_{i-1}. In line 4, we have $F[i] = F[i-1] + F[i-2]$. Using the invariant, $F[i-1] + F[i-2] = F_{i-1} + F_{i-2} = F_i$. At the end of the loop, i is set to $i' = i+1$ and we have $F[i] = F_i = F_{i'-1} = F[i'-1]$, maintaining the invariant.

Termination: At termination, $i = n+1$ and the invariant implies that $F[j] = F_j$ for $j = 1, \ldots, n$.

Problem 2. Maximum Subsequence Sum (8 Marks)

Given an array $A[1..n]$ of positive and negative numbers, we would like to find a subarray $A[i..j]$ such that the sum of the elements in the subarray $\sum_{k=i}^{j} A[k]$ is maximized. Design a **divide and conquer algorithm** for solving this problem. For simplicity, you can return the sum (called the maximum subsequence sum) without returning the index i and j. Give pseudocode, briefly justify your divide and combine steps, and give the asymptotic runtime of your algorithm.

Solution:

We first solve two subproblems of finding the maximum subsequence sum in $A[low\,..\left\lfloor\frac{low+high}{2}\right\rfloor]$ and $A[\left\lfloor\frac{low+high}{2}\right\rfloor+1\,..\,high]$. This forms two candidate solutions for the maximum subsequence sum. We then need to compare the two solutions with the maximum subsequence sum of the subarray that starts in the left subarray and ends in the right subarray – this can be computed in linear time. The pseudocode is shown below.

```
MaxSubSum(A, low, high)
if low == high
    then return A[low]
mid = ⌊(low+high)/2⌋
leftMaxSum = MaxSubSum(A, low, mid)
rightMaxSum = MaxSubSum(A, mid+1,high)
crossMaxSum = MaxCrossSum(A, low, high, mid)
return (leftMaxSum, rightMaxSum, crossMaxSum)
```

The procedure MaxCrossSum computes the maximum subsequence sum that starts in the left subarray and ends in the right subarray. The left end of the subarray can be found in linear time by computing the running sum of array elements from *mid* down to *low* and updating the largest sum that ends at *mid*. The right end of the subarray can be found in a similar way by computing a running sum starting from *mid*+1 to *high*.

```
MaxCrossSum(A, low, high, mid)
leftRunningSum = 0
leftMax = -∞
for i = mid downto low
    leftRunningSum = leftRunningSum + A[i]
    if leftRunningSum > leftMax
        leftMax = leftRunningSum
rightRunningSum = 0
rightMax = -∞
for i = mid+1 to high
    rightRunningSum = rightRunningSum + A[i]
    if rightRunningSum > rightMax
        rightMax = rightRunningSum
return leftMax + rightMax
```

There are two subproblems of half the size and the combine time is $\theta(n)$ giving the recurrence $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$. We use Master theorem case 2. To verify the conditions for case 2, note that $n = \Theta(n^{\lg 2})$, giving $T(n) = \Theta(n \lg n)$.

Comment: It is possible to solve this problem in linear time by using other methods, e.g. by using dynamic programming.

Problem 3. Expected Running Time of Insertion Sort (10 Marks)

Consider the insertion sort algorithm as described below.

```
INSERTION-SORT(A)
1. for j = 2 to A.length
2.     key = A[j]
3.     // Insert A[j] into sorted seq A[1 .. j-1]
4.     i = j –1
5.     while i > 0 and A[i] > key
6.         A[i+1] = A[i]
7.         i = i –1
8.   A[i+1] = key
```

We will compute the running time of insertion sort by counting the number of times array elements are shifted forward as done in Line 6 of the pseudocode.

Assume that the input is a random permutation of n distinct numbers.

a) Let X be the total number of shifts done when sorting the array. Let X_j be the number of times that the elements are shifted in iteration j. Write the expression of X in terms of X_j.

   Solution:

$$X = \sum_{j=2}^{n} X_j$$

b) Let X_{ij} be the indicator random variable for the event that $A[j] < A[i]$ for the input array A. Write the expression for X_j in terms of X_{ij}, where X_j is defined in part (a).

   Solution:

$$X_j = \sum_{i=1}^{j-1} X_{ij}$$

c) Argue that $P(X_{ij}=1) < ½$ and use that to provide an upper bound for $E[X_j]$.

   Solution:

   Under random permutation of n distinct numbers, it is equally likely for $A[j] < A[i]$ and $A[j] > A[i]$ to happen. Hence, $P(X_{ij} = 1) = ½$.

   Note: The question asks for argument that $P(X_{ij}=1) < ½$. This is a bug (I overlooked the assumption that the numbers are distinct). As this part of the question is problematic, the 1 mark allocated to it is given automatically to all students. The other two marks for the for $E[X_j]$ is graded normally.

Using linearity of expectation,

$$E[X_j] = \sum_{i=1}^{j-1} E[X_{ij}]$$

$$= \frac{(j-1)}{2}.$$

d) Using the results above, give an upper bound for $E[X]$.

Solution:

$$\mathrm{E}[X] = \sum_{j=2}^{n} E[X_j]$$

$$= \sum_{j=2}^{n} \frac{(j-1)}{2} = \frac{(n-1)n}{4} = O(n^2)$$

END OF ASSESSMENT

Scratch Paper

Scratch Paper